Versión 1.0

Paradigmas de Programación

Bases de Programación

ING. DAVID CECCHI
ING. JACOBO DOMINGUEZ
ING. NICOLÁS TIZZANO

UNIVERSIDAD TECNOLOGICA NACIONAL
Facultad Regional San Nicolás
Ingeniería en Sistemas

Repaso de Algoritmia

Un algoritmo representa una secuencia de pasos, que al ser ejecutados resuelven un problema dado. Los pasos conforman una secuencia y **no un conjunto** porque **el orden en que se lleven a cabo es extremadamente importante**.
Ejemplificando, para fabricar una casa es necesario construir los cimientos, las paredes y el techo; en ese orden, ya que en cualquier otro orden de ejecución no se obtendrá como resultado una casa. No es posible comenzar a colocar el techo si antes no contamos con los cimientos y las paredes.
Otro ejemplo a plantear es la receta para elaborar una torta, los ingredientes deben incorporarse en cierto orden y en una determinada cantidad. Cualquier alteración en los pasos implicaría no obtener los resultados esperados.
La dificultad en elaborar un algoritmo parte del hecho que, como personas, estamos acostumbrados precisamente a comunicarnos con pares y damos por sentado acciones que en realidad son muy complejas. Ejemplificando, si estamos en casa y solicitamos a alguien que nos sirva un vaso con agua, seguramente obtendremos el servicio solicitado; sin necesidad de dar más explicaciones. En cambio, un programador, no se comunica con personas, sino con máquinas y allí surge el inconveniente.
Una maquina tiene una gran ventaja que es su velocidad, puede ejecutar millones de operaciones por segundo de manera repetitiva sin agotarse (siempre con el mismo grado de precisión), pero tiene así mismo una gran desventaja: solo entiende instrucciones muy básicas. Por lo tanto, es imprescindible comunicarle cualquier acción con un nivel de detalle minucioso. Es decir, si debemos programar un robot humanoide con la capacidad de responder a la solicitud de servir un vaso con agua y partimos del supuesto que está capacitado para desplazarse y asir cosas; deberíamos detallarle: cuantos pasos (distancia) debe desplazarse y en qué dirección debe ir a buscar el vaso, donde se encuentra la fuente de agua, cual es la canilla que debe abrir, en qué sentido la debe girar, la fuerza que debe aplicar para no dañarla, cantidad de agua que debe servir, etc. Por lo cual, un problema muy simple para un humano (traer un vaso con agua) pueden implicar cientos de pequeñas acciones al momento de replicar la tarea en una máquina. Incluso, cada acción puede involucrar acciones (bases) de menor nivel: lograr desplazamiento, mantener el equilibrio, mover extremidades, etc.
En consecuencia, un programador o desarrollador, debe ser capaz de poder identificar la problemática y desarrollar una solución eficaz (logra el objetivo) y eficiente (uso óptimo de los recursos: minimizar la cantidad de líneas de código, reducir el tiempo de respuesta, minimizar el uso de memoria, etc.); para ello se requiere capacidad de lógica y abstracción. Luego deberá desglosar la solución hallada en una secuencia de acciones muy básicas (aquellas acciones que una máquina puede entender). En este proceso reside la complejidad de la programación, el solo hecho de aprender un lenguaje no hace a un buen programador.
Ahora bien, ¿cuáles son esas acciones básicas que una máquina es capaz de entender?, las podemos agrupar en:

- ***Elementales***: **INGRESAR**, permite recibir datos del exterior los cuales pudieron ser ingresados desde un teclado, mouse, sensor, etc. **CALCULAR**, realiza una operación aritmética. **MOSTRAR**, presenta información por pantalla (visual), impresora (visual), parlante (sonora), movimiento (física), etc.

- ***De Decisión***: **CONDICIONAL** y **SELECCIÓN MULTIPLE**, permite a la máquina comparar valores y tomar una decisión al respecto. Las condiciones a analizar (expresadas respetando la lógica booleana, solo admiten como respuesta un verdadero o falso) y las acciones que se deben ejecutar, las fijará el programador. Si la maquina arroja resultados incorrectos o presenta errores en el funcionamiento será responsabilidad del programador. *Recuerde que* ***una máquina solo cumple órdenes, no llega a entender lo que ella misma realiza, no mide consecuencia de sus actos, no tiene conciencia, moral ni saber***.

- ***De Repetición***: **PARA** y **MIENTRAS**, permite repetir una o varias acciones una cantidad conocida (PARA) o desconocida (MIENTRAS). Si debido a la cantidad de repeticiones permitidas la maquina cae en un ciclo infinito de repetición, también será responsabilidad del programador, por las mismas razones expuestas en el punto anterior.

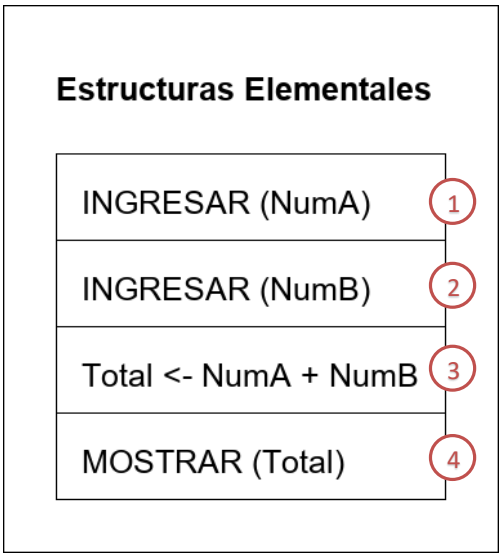
Estructuras Algorítmicas

En los ejemplos presentados a continuación, con el objetivo de facilitar la interpretación, se plantean conversaciones entre dos amigos, donde Juan actúa en el papel de Usuario y Luis en el papel de Aplicación.

Para ejemplificar la utilización de **estructuras elementales**, se plantea la siguiente situación:

Juan: Buenos días Luis, ¿podrías decirme cuál es el resultado de sumar dos números?.
Luis: Buenos días Juan, ¿Cuál es el primer número?.
Juan: 1778.
Luis: ¿Cuál es el segundo número?.
Juan: 2292.
Luis: Dame un segundo para pensar (sumar).
Luis: Listo!, el resultado es 4070.

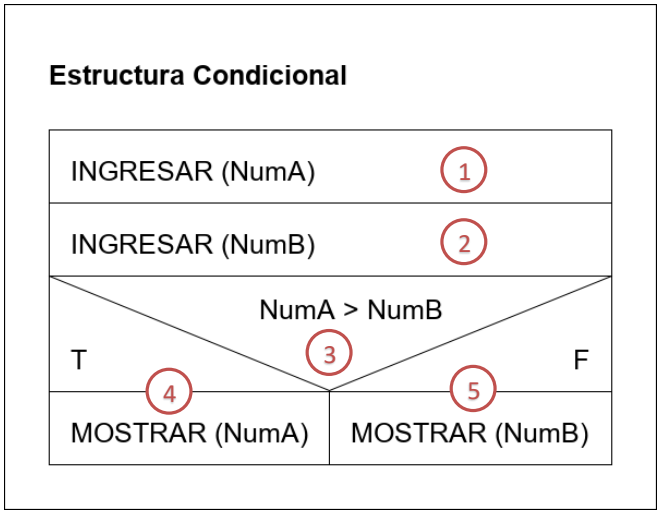
Si debiéramos plantear un algoritmo que realice la tarea de Luis, es decir, ser capaz de sumar dos números al azar, podríamos expresarlo de la siguiente manera:



1. Luis, solicita conocer el primer número a procesar y lo memoriza.
2. Luis, solicita conocer el segundo número a procesar y lo memoriza.
3. Luis, realiza la suma de los números recibidos y memoriza el resultado obtenido.
4. Luis, da a conocer el resultado.

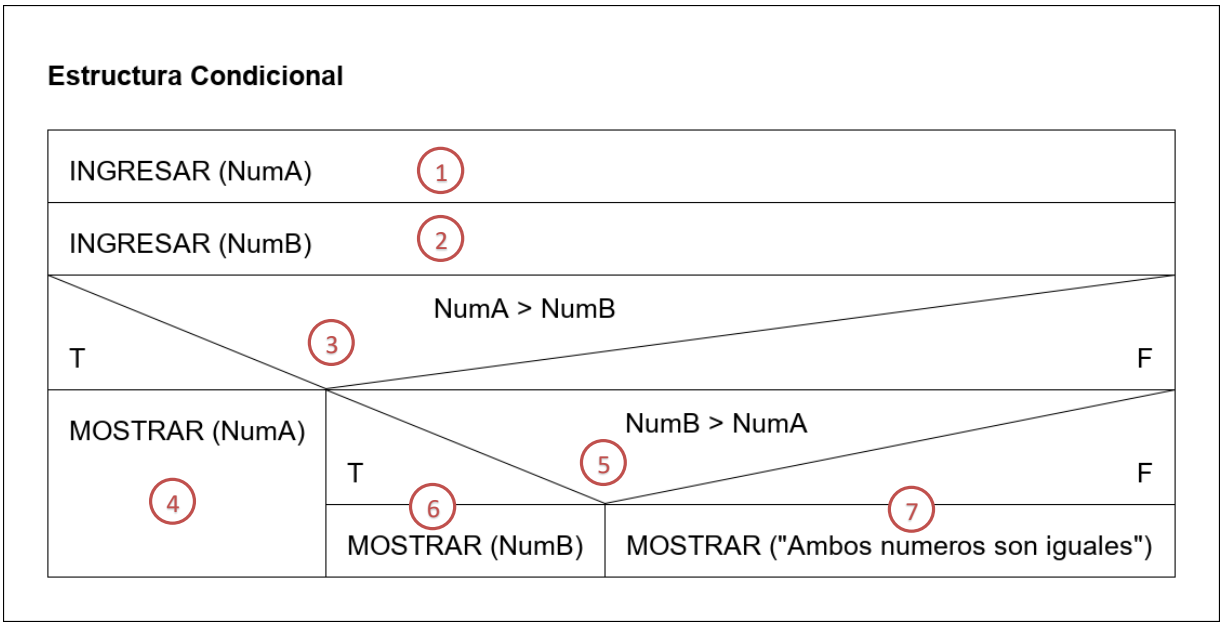
Para ejemplificar la utilización de la **estructura condicional**, se plantea la siguiente situación:

Juan: Buenos días Luis, ¿podrías decirme cuál de dos números es el mayor?
Luis: Buenos días Juan, ¿Cuál es el primer número?.
Juan: 1778.
Luis: ¿Cuál es el segundo número?.
Juan: 2292.
Luis: Dame un segundo para pensar (comparar).
Luis: Listo!, el mayor es 2292.



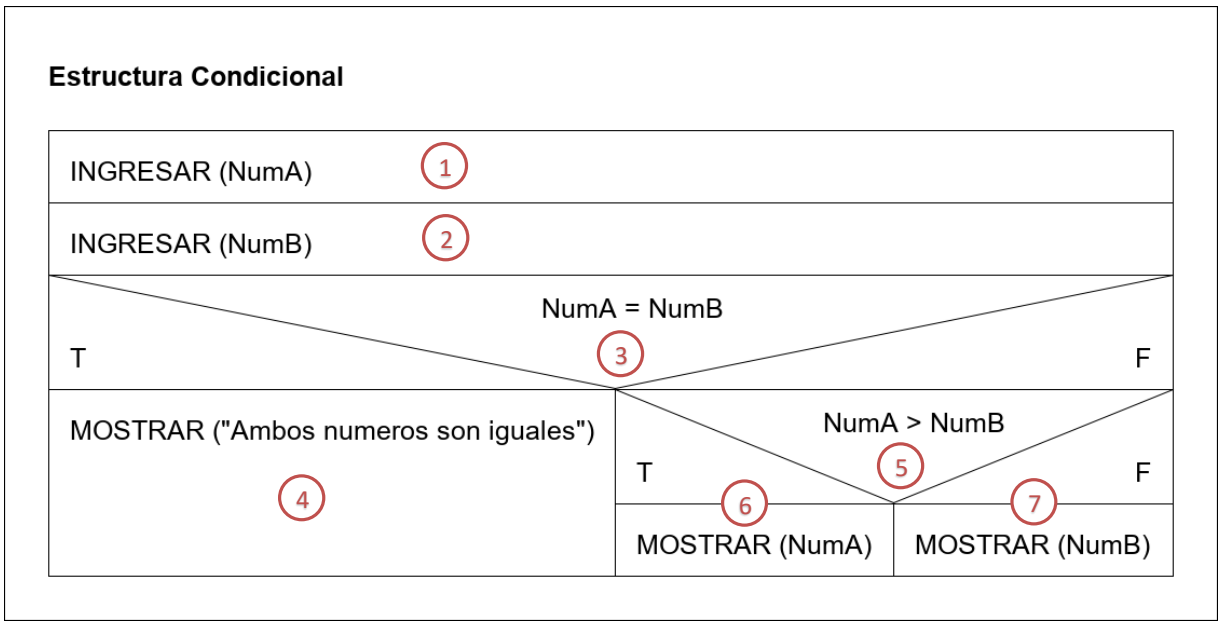
1. Luis, solicita conocer el primer número a procesar y lo memoriza.
2. Luis, solicita conocer el segundo número a procesar y lo memoriza.
3. Luis, analiza los números recibidos preguntándose si el primer número es mayor al segundo número.
4. Luis, dará a conocer el valor del primer número si la pregunta tuvo respuesta afirmativa.
5. Luis, dará a conocer el valor del segundo número si la pregunta tuvo respuesta negativa.

La respuesta brindada puede prestarse a confusión en caso que ambos números sean iguales. En esta situación, el algoritmo anterior diría que el mayor es el valor contenido en NumB siendo que ambos son idénticos. Para solucionar este inconveniente, se presenta una versión mejorada del algoritmo:



1. Luis, solicita conocer el primer número a procesar y lo memoriza.
2. Luis, solicita conocer el segundo número a procesar y lo memoriza.
3. Luis, analiza los números recibidos preguntándose si el primer número es mayor al segundo número.
4. Luis, dará a conocer el valor del primer número si la pregunta anterior tuvo respuesta afirmativa.
5. Luis, al obtener una respuesta negativa, analizará nuevamente los números recibidos preguntándose si el segundo número es mayor al primer número.
6. Luis, dará a conocer el valor del segundo número si la pregunta tuvo respuesta afirmativa.
7. Luis, dirá que “Ambos números son iguales” si la pregunta tuvo respuesta negativa.

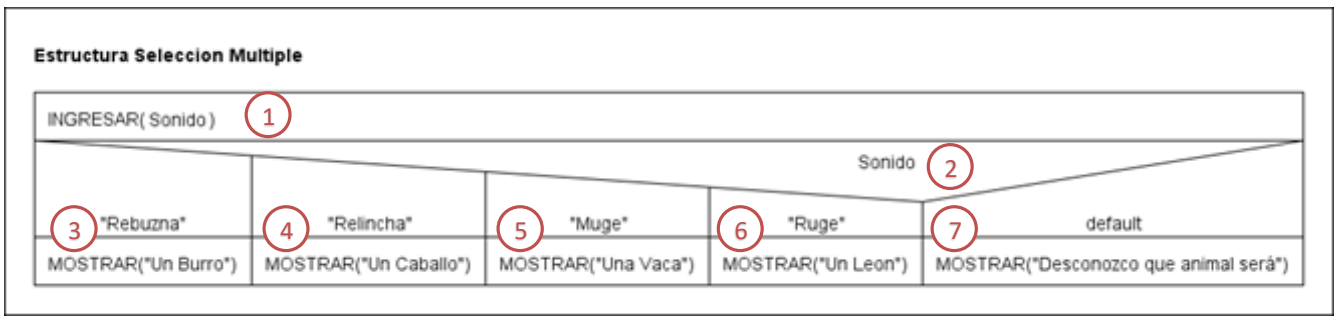
Generalmente, existen múltiples algoritmos que resuelven un problema dado, por ejemplo, el siguiente algoritmo también funciona correctamente:



1. Luis, solicita conocer el primer número a procesar y lo memoriza.
2. Luis, solicita conocer el segundo número a procesar y lo memoriza.
3. Luis, analiza los números recibidos preguntándose si los números son iguales.
4. Luis, dirá que “Ambos números son iguales” si la pregunta tuvo respuesta afirmativa.
5. Luis, al obtener una respuesta negativa, analizará nuevamente los números recibidos preguntándose si el primer número es mayor al segundo número.
6. Luis, dará a conocer el valor del primer número si la pregunta tuvo respuesta afirmativa.
7. Luis, dará a conocer el valor del segundo número si la pregunta tuvo respuesta negativa.

Para ejemplificar la utilización de la **estructura de selección múltiple**, se plantea la siguiente situación:

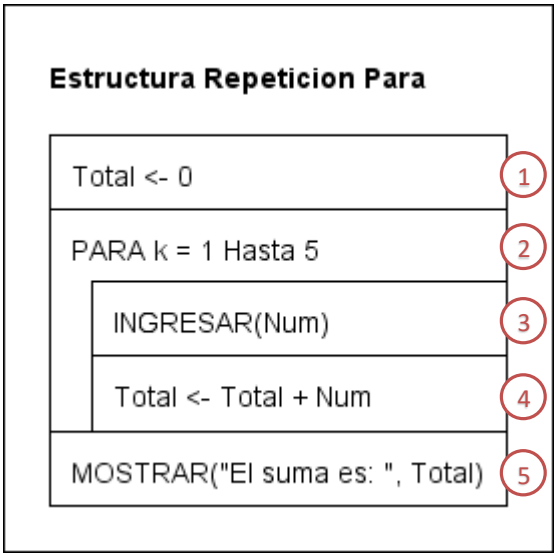
Juan: Buenos días Luis, si te nombro el sonido que realiza un animal, ¿podrías decirme a qué animal pertenece?
Luis: Buenos días Juan, ¿Cuál es el sonido?.
Juan: Ruge.
Luis: Dame un segundo para pensar (comparar).
Luis: Listo!, pertenece a un león.



1. Luis, solicita conocer el sonido a procesar y lo memoriza.
2. Luis, analiza el sonido recibido preguntándose si coincide con alguno de los que él conoce.
3. Luis, dirá “Un Burro” si el sonido coincide con “Rebuzna”.
4. Luis, dirá “Un Caballo” si el sonido coincide con “Relincha”.
5. Luis, dirá “Una Vaca” si el sonido coincide con “Muge”.
6. Luis, dirá “Un León” si el sonido coincide con “Ruge”.
7. Luis, dirá “Desconozco que animal será” si el sonido no coincide con ninguno de los conocidos por él.

Para ejemplificar la utilización de la **estructura de repetición Para**, se plantea la siguiente situación:

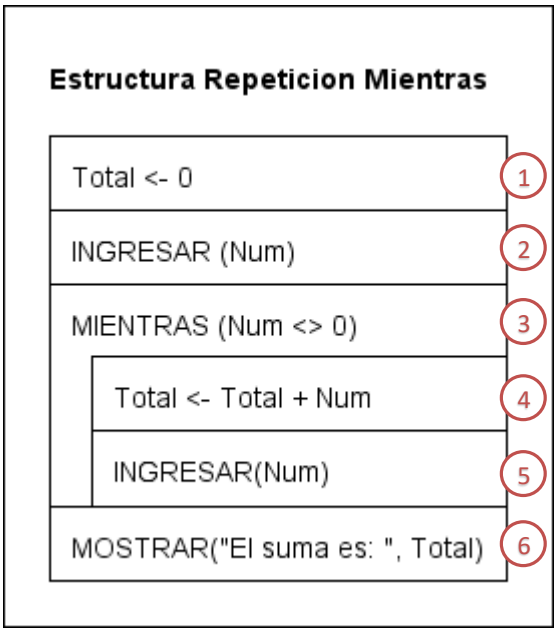
Juan: Buenos días Luis, si te dicto 5 números, me podrías decir cuánto suman?
Luis: Buenos días Juan, seguro! Uno a uno por favor.
Juan: 17.
Juan: 23.
Juan: 10.
Juan: 35.
Juan: 21.
Luis: Listo!, el total es 106.



1. Luis, inconscientemente siempre empezará a acumular desde cero (no tiene números previos).
2. Luis, se prepara para recibir el primer número.
   3. Luis, recibe el primer número y lo memoriza.
   4. Luis, acumula el primer número memorizado.
2. Luis, se prepara para recibir el segundo número.
   3. Luis, recibe el segundo número y lo memoriza.
   4. Luis, acumula el segundo número memorizado.
2. Luis, se prepara para recibir el tercer número.
   3. Luis, recibe el tercer número y lo memoriza.
   4. Luis, acumula el tercer número memorizado.
2. Luis, se prepara para recibir el cuarto número.
   3. Luis, recibe el cuarto número y lo memoriza.
   4. Luis, acumula el cuarto número memorizado.
2. Luis, se prepara para recibir el quinto número.
   3. Luis, recibe el quinto número y lo memoriza.
   4. Luis, acumula el quinto número memorizado.
5. Luis, dará a conocer el resultado de la suma que fue memorizando.

Para ejemplificar la utilización de la **estructura de repetición Mientras**, se plantea la siguiente situación:

Juan: Buenos días Luis, ¿si te dicto una lista de números, me podrías decir cuánto suman? Finalizaré cuando mencione el cero!
Luis: Buenos días Juan, seguro! Uno a uno por favor.
Juan: 17.
Juan: 23.
Juan: 7.
Juan: 0.
Luis: Listo!, el total es 47.



1. Luis, inconscientemente siempre empezará a acumular desde cero (no tiene números previos).
2. Luis, recibe el primer número y lo memoriza.
3. Luis, analiza el primer número recibido preguntándose si el número no es cero.
    4. Luis, acumula el primer número memorizado.
    5. Luis, recibe el segundo número y lo memoriza.
3. Luis, analiza el segundo número recibido preguntándose si el número no es cero.
    4. Luis, acumula el segundo número memorizado.
    5. Luis, recibe el tercer número y lo memoriza.
3. Luis, analiza el tercer número recibido preguntándose si el número no es cero.
    4. Luis, acumula el tercer número memorizado.
    5. Luis, recibe el cuarto número y lo memoriza.
3. Luis, analiza el cuarto número recibido preguntándose si el número no es cero, al tener una respuesta negativa deja de solicitar números.
6. Luis, dará a conocer el resultado de la suma que fue memorizando.

¿Cómo se obtiene una aplicación?

Con el objetivo de dar un marco a los conceptos que se deben abordar, se presenta una serie de pasos simplificados necesarios para obtener una aplicación funcional.

1. **Relevamiento:** consiste en la identificación de los requerimientos, entendimiento del contexto y recopilación de datos (discriminando aquellos datos útiles para nuestros fines, de aquellos datos superfluos). Es considerada la etapa más importante, ya que las restantes se basan sobre ella. Si el problema no está perfectamente definido y comprendido, cualquier solución que se obtenga será incorrecta. Esta afirmación radica en que se estará resolviendo un problema diferente al planteado por el cliente, generando la perdida de interés y, en consecuencia, desencadenando la no aceptación por parte del interesado. En referencia al desarrollador, se generan pérdidas económicas, de tiempo, esfuerzo y malas referencias laborales, entre otros aspectos negativos.

2. **Diseño de solución:** a partir de un análisis profundo sobre la información obtenida en la etapa anterior, se deberá definir el paradigma a implementar, el algoritmo (lógica) a desarrollar para alcanzar la solución, seleccionar el lenguaje de programación a utilizar (en consonancia con el paradigma escogido), arquitectura a implementar, recursos necesarios, etc.

3. **Desarrollo:** en este momento, las acciones que conforman al algoritmo obtenido en la etapa anterior, serán "traducidas" por el programador a órdenes propias del lenguaje de programación seleccionado obteniendo así el código fuente. Luego, a través de un proceso denominado compilación, el código fuente será convertido a código de máquina, necesario para que el autómata pueda ejecutar las acciones mencionadas. Es decir, al final de ésta etapa se obtendrá la aplicación objetivo.

4. **Prueba:** la finalidad de esta etapa es encontrar y solucionar los posibles errores que presenta la aplicación desarrollada. Para ello se intentará testear la misma con datos reales o ficticios. Los errores que pueden hallarse se clasifican en:

    4.1. **Errores de sintaxis o de compilación:** Es una violación de las reglas del lenguaje de programación. Son más fáciles de corregir, ya que son detectados por el compilador (posible error de escritura), el cual dará información sobre el lugar donde fue hallado y la naturaleza de cada uno de ellos mediante un mensaje aclaratorio.

    4.2. **Errores de Ejecución:** Se deben generalmente a operaciones no permitidas como dividir por cero, leer un dato no numérico en una variable numérica, exceder un rango de valores permitidos, etc. Se detectan porque se produce una parada anormal del programa durante su ejecución.

    4.3. **Errores de Lógica:** Corresponden a la obtención de resultados incorrectos y la única manera de detectarlos es realizando suficientes pruebas del programa. Son de difícil detección.

    4.4. **Errores de Especificación:** Es el peor tipo de error y el más difícil de corregir. Se deben a una falla en la etapa de relevamiento debido a una mala comunicación con usuario y se detectan cuando ya se ha concluido el diseño y con frecuencia, la instalación del programa. Suelen implicar repetir gran parte del trabajo realizado.

5. **Documentación:** Consiste en describir por escrito a nivel técnico los procedimientos relacionados con el desarrollo de la aplicación y su modo de uso (manual del usuario).

6. **Mantenimiento:** Es el paso final del desarrollo del software. Alrededor del 75% del costo total del ciclo de vida de una aplicación se destina al mantenimiento. El propósito del mantenimiento es garantizar

que las aplicaciones en uso estén libres de errores de operación, sean eficientes, efectivas y se adapten a las nuevas necesidades.

Definiciones relacionadas

Lenguaje de programación
Como puede inferirse, representa un idioma que permite la comunicación entre un programador y un autómata. Formalmente, es un conjunto de elementos, sentencias y reglas (sintaxis) que deben respetarse para generar el código fuente.

Sentencia
Es una orden escrita en un lenguaje de programación determinado. Las sentencias varían de un lenguaje a otro.

Código Fuente
Es un texto escrito en un determinado lenguaje de programación que representa (da forma) al algoritmo desarrollado previamente.

Compilación
Una vez generado el código fuente, deberá pasar por un proceso denominado compilación (traducción) que convertirá el código fuente expresado en un lenguaje de programación (entendible por el programador), a código de máquina (ejecutable) que se encontrará expresado en lenguaje de máquina (entendible por el autómata). El código ejecutable obtenido, será aquella aplicación que utilizará el usuario.

Como síntesis, el código fuente es el conjunto de sentencias (las cuales deben respetar las exigencias semánticas y de sintaxis del lenguaje de programación utilizado), que compiladas constituirán la aplicación. Dicha aplicación será interpretada por el autómata quien, una vez que recibe la orden de inicio, comienza a ejecutarlas, llevando a cabo la secuencia de acciones que conforman el algoritmo que resuelve el problema.

Pautas al Programar

Al desarrollar una aplicación, no solo se debe pretender cumplir con los requerimientos identificados durante las reuniones con el cliente, también se deberá pensar a futuro, es decir tener presente que todo software durante su ciclo de vida es susceptible de sufrir modificaciones. Estas modificaciones pueden tener su raíz en cuestiones tales como: nuevos requerimientos o cambios a los ya existentes, mejoras en el algoritmo haciéndolo más eficiente en el uso de los recursos (ej. tiempo, memoria), etc. Estos factores, por lo tanto, obligan a ampliar y/o retocar el código fuente.
Transcurrido un período considerable desde la finalización del programa, toda modificación obligará a la reinterpretación del código. Durante su generación, cada línea de código fuente resulta prácticamente obvia para el desarrollador por encontrarse inmerso en su elaboración, pero al transcurrir un tiempo prolongado desde su creación los detalles del desarrollo son olvidados dificultando enormemente la tarea de análisis. Dicha reinterpretación aumentará en complejidad si el código fue elaborado por otro individuo, con su propia lógica y costumbres de programación (herramientas, técnicas, estilos, etc.).
En consecuencia, en la etapa de desarrollo se buscará que el código generado cumpla con los siguientes requerimientos: *claridad, legibilidad y modificabilidad,* los cuales facilitaran su entendimiento, permitiendo de esta manera reducir los tiempos de desarrollo y mantenimiento, dando lugar a la generación de aplicaciones más robustas.

a. ***Claridad:*** implica que la resolución algorítmica sea sencilla, que esté correctamente estructurada, resultando un algoritmo de fácil compresión.

b. ***Legibilidad:*** significa que en la codificación se recurra a ***nombres adecuados*** en lo que respecta a variables, funciones, procedimientos, etc., ***comentarios aclaratorios*** y una correcta ***diagramación*** del texto.

c. ***Modificabilidad:*** cualquier cambio que sea necesario incorporar en alguna de sus partes, no obligue a realizar cambios y/o controles en todo el programa, sino limitarlo solo al módulo de interés.

Conceptos comunes entre lenguajes de programación

Comentarios
Se trata de texto que es ignorado por el compilador al traducir el código. Esas líneas se utilizan para documentar el programa. Esta labor de documentación es fundamental ya que luego de un tiempo el código desarrollado no se comprende con facilidad, ni siquiera por el autor del mismo. La tarea de comentar es aún más importante cuando el código fuente va a ser tratado por otras personas del equipo de desarrollo.

Palabras reservadas
Se denominan así a palabras que en el lenguaje de programación utilizado tienen un significado concreto para el compilador. Por tanto, no es posible utilizar estas palabras como identificadores.

Identificadores
Son los nombres que damos a las constantes, variables y funciones. Lógicamente no pueden coincidir con las palabras reservadas. Existen lenguajes que distinguen entre mayúsculas y minúsculas, en ellos, se debe dedicar especial atención al definir los identificadores (es decir: nombre, Nombre y NOMBRE son tres identificadores distintos).
En la mayoría de los lenguajes, los identificadores, deben de cumplir las siguientes reglas:

- Deben comenzar por una letra.
- Sólo se admiten letras (del abecedario inglés, no se admite ni la ñ ni la tilde ni la diéresis), números y el carácter de subrayado.

Constantes
Una constante es un dato cuyo valor NO puede cambiar durante la ejecución del programa. Recibe un valor en el momento de la compilación y este permanece inalterado durante todo el programa.
Tienen varios usos: miembro en una expresión, valor de inicialización para variables, etc.

Variables
Una variable es una celda o dirección de memoria, la cual contiene un único dato a la vez y su valor puede cambiar durante la ejecución del programa.
Se distinguen cuatro etapas fundamentales en la vida de una variable:

1. **Declaración:** es la primera fase en la vida de cualquier variable. La declaración se realiza en una sección particular del programa. Se deberá indicar en este momento el **Tipo de Dato** a almacenar en ella, es decir, se expresará si la misma permitirá contener únicamente valores numéricos enteros, decimales, cadenas de texto, un caracter, un dato booleano, etc.
2. **Inicialización:** implica darle un valor inicial. Se realiza para evitar posibles errores en tiempo de ejecución. Dependiendo del lenguaje utilizado, una variable puede contener un valor indeterminado después de declararla.
3. **Utilización:** se refiere a la aplicación de la variable dentro del bloque en el que se ha declarado.
4. **Liberación:** cuando finaliza el bloque en el que fue declarada, la variable muere. Es decir, se libera el espacio que ocupa en memoria, para que el mismo pueda ser utilizado por otro programa (proceso). Al fallecer, no se la podrá volver a utilizar.

Ámbito o Alcance de una Variable

Toda variable tiene un ámbito. Es el área de código en la cual la variable es reconocida como tal y puede ser utilizada.

1. **Local:** son variables que se declaran dentro de un bloque. Se entiende por bloque, a una sección de código. Dependiendo del lenguaje utilizado, un bloque puede encontrarse delimitado por llaves "{…}". Al finalizar el bloque la variable es eliminada.
2. **Global:** son variables que pueden ser utilizadas en cualquier bloque de código. Para que una variable sea global basta con declararla fuera de todo bloque. Normalmente se declaran antes de la primera función en la cual es utilizada. No es aconsejable el uso/abuso de variables globales; usarlas causa malos hábitos.
   Al implementar variables globales se puede plantear una inquietud: si un bloque cuenta con una variable local cuyo identificador coincide con el identificador de una variable global, ¿qué variable utilizará el bloque: la local o la global? La respuesta es: siempre se tomará la variable declarada más localmente.

No es posible declarar en el mismo bloque dos variables con el mismo nombre (identificador), pero sí es posible definir dos variables con el mismo nombre si se encuentran en bloques distintos.

Anexo: Sintaxis en C#

El presente anexo tiene por objetivo ser una referencia rápida sobre temas elementales que serán requeridos en el transcurso de la cátedra, no pretende abarcar de manera exhaustiva las características del lenguaje.

Tipos de Datos

Tipo	Descripción	Bits	Rango de valores	Alias
SByte	Bytes con signo	8	[-128, 127]	**sbyte**
Byte	Bytes sin signo	8	[0, 255]	**byte**
Int16	Enteros cortos con signo	16	[-32.768, 32.767]	**short**
UInt16	Enteros cortos sin signo	16	[0, 65.535]	**ushort**
Int32	Enteros	32	[-2.147.483.648, 2.147.483.647]	**int**
UInt32	Enteros sin signo	32	[0, 4.294.967.295]	**uint**
Int64	Enteros largos	64	[-9.223.372.036.854.775.808, 9.223.372.036.854.775.807]	**long**
UInt64	Enteros largos sin signo	64	[0-18.446.744.073.709.551.615]	**ulong**
Single	Reales con 7 dígitos de precisión	32	$[1,5\times10^{-45} - 3,4\times10^{38}]$	**float**
Double	Reales de 15-16 dígitos de precisión	64	$[5,0\times10^{-324} - 1,7\times10^{308}]$	**double**
Decimal	Reales de 28-29 dígitos de precisión	128	$[1,0\times10^{-28} - 7,9\times10^{28}]$	**decimal**
Boolean	Valores lógicos	32	true, false	**bool**
Char	Caracteres Unicode	16	[‘\u0000’, ‘\uFFFF’]	**char**
String	Cadenas de caracteres	Variable	El permitido por la memoria	**string**
Object	Cualquier objeto	Variable	Cualquier objeto	**object**
DateTime	Representa un instante de tiempo	64	01/01/0001 00:00:00, 31/12/9999 23:59:59	**DateTime**

Operadores

Operadores Matemáticos	Significado	Operadores Comparación	Significado	Operadores Lógicos	Significado
+	Suma	>	Mayor que	&&	AND
-	Resta	>=	Mayor o igual que	||	OR
*	Producto	<	Menor que	!	NOT
/	División	<=	Menor o igual que		
%	resto (módulo)	==	Igual que		
++	Incremento	!=	Distinto de		
--	Decremento				

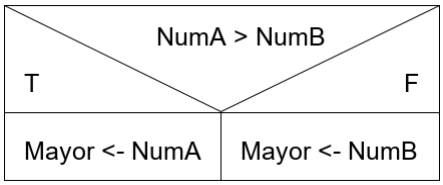
Estructuras de Control

<table>
<tr><th>Estructura</th><th>Sintaxis</th></tr>
<tr><td rowspan="3">Condicional</td><td>

```
if (Condicion)
{
   Instrucciones a ejecutar cuando la
   condicion resulta verdadera
}
else
{
   Instrucciones a ejecutar cuando la
   condicion resulta falsa
}
```

</td></tr>
<tr><td>
<table>
<tr><th>Ejemplo</th><th>Estructura Asociada</th></tr>
<tr><td>

```
if (NumA > NumB)
{
   Mayor = NumA;
}
else
{
   Mayor = NumB;
}
```

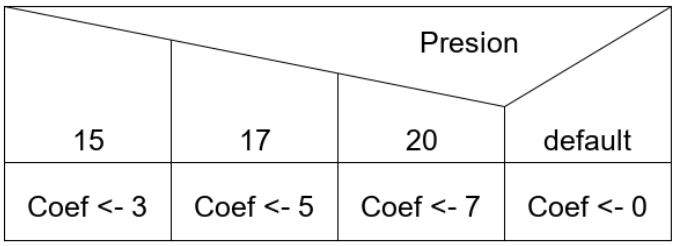
</td><td> 
</td></tr>
</table>
</td></tr>
<tr><td>Selección Múltiple</td><td>

```
switch (Expresion)
{
   case Valor1:
      {
         Instrucciones a ejecutar
         cuando la Expresion
         coincida con Valor1
         break;
      }
   case ValorN:
      {
         Instrucciones a ejecutar
         cuando la Expresion
         coincida con ValorN
         break;
      }
   default:
      {
         Instrucciones a ejecutar
         cuando la Expresion
         no coincida con ningun Case
         break;
      }
}
```

</td></tr>
</table>

<table>
<tr><td rowspan="2"></td><td>Ejemplo</td><td>Estructura Asociada</td></tr>
<tr><td>

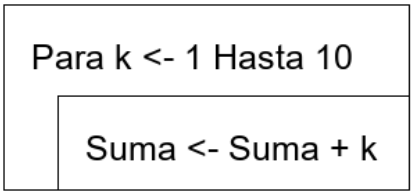
```
switch (Presion)
{
   case 15:
      {
         Coef = 3;
         break;
      }
   case 17:
      {
         Coef = 5;
         break;
      }
   case 20:
      {
         Coef = 7;
         break;
      }
   default:
      {
         Coef = 0;
         break;
      }
}
```

</td><td> 
</td></tr>
<tr><td rowspan="3">Para</td><td colspan="2">Sintaxis</td></tr>
<tr><td colspan="2">

```
for (Inicio;Condicion;Incremento)
{
   Instrucciones a ejecutar en cada
   ciclo
}
```

</td></tr>
<tr><td>Ejemplo</td><td>Estructura Asociada</td></tr>
<tr><td>

```
for (k = 1; k <= 10; k++)
{
   Suma = Suma + k;
}
```

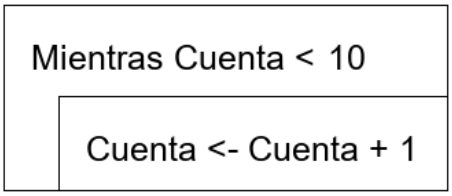
</td><td> 
</td></tr>
<tr><td rowspan="2">Mientras</td><td colspan="2">Sintaxis</td></tr>
<tr><td colspan="2">

```
while (Condicion)
{
   Instrucciones a ejecutar en cada
   ciclo
}
```

</td></tr>
</table>

<table>
<tr><td></td><td>Ejemplo</td><td>Estructura Asociada</td></tr>
<tr><td></td><td>

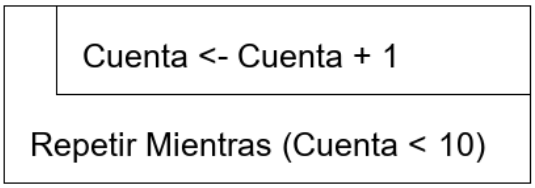
```
while (Cuenta < 10)
{
   Cuenta = Cuenta + 1;
}
```

</td><td> 
</td></tr>
<tr><td rowspan="4">Repetir Mientras</td><td colspan="2">Sintaxis</td></tr>
<tr><td colspan="2">

```
do
{
   Instrucciones a ejecutar en cada
   ciclo
}
while (Condicion);
```

</td></tr>
<tr><td>Ejemplo</td><td>Estructura Asociada</td></tr>
<tr><td>

```
do
{
   Cuenta = Cuenta + 1;
}
while (Cuenta < 10);
```

</td><td> 
</td></tr>
<tr><td rowspan="4">Para Cada</td><td colspan="2">Sintaxis</td></tr>
<tr><td colspan="2">

```
foreach (Tipo Objeto in Arreglo/Coleccion)
{
   Instrucciones a ejecutar en cada
   ciclo
}
```

</td></tr>
<tr><td>Ejemplo</td><td></td></tr>
<tr><td>

```
foreach (int k in vector)
{
   Suma = Suma + k;
}
```

</td><td></td></tr>
</table>

Declaración de Variables

Objetivo	Sintaxis
Declarar una variable.	TipoDeDato NombreDeVariable;
Declarar un conjunto de variables de un mismo tipo de dato.	TipoDeDato NombreDeVariable1,NombreDeVariable2,NombreDeVariable3;
Declarar una variable asignándole un valor inicial.	TipoDeDato NombreDeVariable = ValorInicial;
Declarar un vector sin especificar la cantidad de filas a contener.	TipoDeDato[] NombreDeVariable;
Declarar un vector especificando la cantidad de filas a contener.	TipoDeDato[] NombreDeVariable = new TipoDeDato[CantFilas];
Declarar un vector especificando el contenido del mismo.	TipoDeDato[] NombreDeVariable = {Valor1,Valor2,Valor3};
Declarar una matriz de 2 dimensiones sin especificar la cantidad de filas y columnas a contener.	TipoDeDato[,] NombreDeVariable;
Declarar una matriz de 2 dimensiones especificando la cantidad de filas y columnas a contener.	TipoDeDato[,] NombreDeVariable = new TipoDeDato[CantFilas,CantCols];
Declarar una matriz de 2 dimensiones de 3x2 especificando el contenido de la misma.	TipoDeDato[,] NombreDeVariable = {{Valor11,Valor12},{Valor21,Valor22},{Valor31,Valor32}};

Modificador Const

Las variables declaradas con la palabra **const** delante del tipo de datos, indican que son de sólo lectura. Es decir, constantes. Las constantes no pueden cambiar de valor, el valor que se asigne en la declaración será el que permanezca (es obligatorio asignar un valor en la declaración). Ejemplo: const float PI=3.141592.

Comentar líneas de código

Con frecuencia es útil incorporar dentro del código fuente comentarios aclaratorios sobre determinadas situaciones a tener en cuenta, recordatorios o temporalmente deshabilitar ciertas líneas de código que no desea que sean compiladas. Para ello existen 2 alternativas:

1. Comentar una única línea: al comienzo de la misma se deben tipear dos barras inclinadas **//**
   Ejemplo:
   //es un comentario y no será compilado

2. Comentar un conjunto de líneas consecutivas (bloque): al comienzo del mismo se debe tipear **/*** y al finalizar ***/**
   Ejemplo:
   /*es un comentario de múltiples líneas
   y no será compilado*/

Funciones y Procedimientos

Un **módulo** es un bloque de código creado para llevar a cabo una determinada tarea. La utilización de módulos dentro del programa permitirá, como su nombre lo indica, llegar al concepto de modularidad. En todo algoritmo se busca alcanzar modularidad, base para lograr la tan deseada modificabilidad. Un módulo correctamente desarrollado no necesita tener conocimiento sobre la lógica implementada dentro de los módulos que invoca (concepto de caja negra). En consecuencia, la solución total (algoritmo principal) quedará compuesta por partes (módulos) independientes, permitiendo prácticamente que cualquier modificación requerida, se logre sin retocar nada más que el módulo involucrado.

Procedimientos: Son aquellos módulos que no devuelven un valor, en su declaración se utiliza la palabra clave **void**. Un procedimiento es utilizado para llevar a cabo una tarea determinada para la cual se considera que el módulo invocante no requerirá información de la operación a realizar. Un ejemplo práctico sería un procedimiento dedicado al blanqueo de pantalla.

Funciones: Son aquellos módulos que devuelven un valor, es decir en la declaración de la función se especifica el tipo de dato (int, char, float, etc) que la misma retornará al módulo invocante. El valor retornado por la función será especificado dentro de la misma a través de la palabra clave **return** seguido por la expresión a retornar, y podrá ser utilizado por el módulo invocante para alcanzar su objetivo. El valor retornado por la función será un valor único, inclusive podría llegar a ser una estructura, pero nunca un arreglo, en todo caso un puntero al mismo (concepto que se detallará más adelante). Un ejemplo práctico es una función dedicada a la obtención del valor absoluto de un número dado.

Procedimiento	

Declaración	void NombreProcedimiento (TipoParam1 NombreParam1, TipoParam2 NombreParam2,….)
	{
	declaracion variables;
	codigo
	}

Procedimiento que presenta un mensaje por pantalla cuyo destinatario debe ser especificado.	

Declaración	static void Saludar(string Usuario)
	{
	Console.WriteLine("Hola {0}.", Usuario);
	}

Utilización	Saludar("Pedro"); Luego de la ejecución de esta línea, se presentará el siguiente mensaje: Hola Pedro.

Función	

Declaración	TipoDatoRetornar NombreFuncion (TipoParam1 NombreParam1, TipoParam2 NombreParam2,….)
	{
	declaracion variables;
	codigo
	return (expresion);
	}

Función que retorna el Valor Absoluto de un número entero.	

Declaración	static int ValorAbsoluto(int Numero)
	{
	if (Numero < 0)
	{
	return (-Numero);
	}
	else
	{
	return (Numero);
	}
	}

Utilización	ValorAbs = ValorAbsoluto(-3); Luego de la ejecución de esta línea, la variable ValorAbs contendrá el valor 3.

Parámetros

Un **parámetro o argumento**, es el elemento por medio del cual se transmiten datos de un módulo a otro.
Los parámetros pueden ser clasificados como:

Por Valor: también conocido como parámetro de solo entrada. Es un parámetro que, en el momento de la invocación del módulo (función/procedimiento), el valor por él recibido es copiado al mismo y por lo tanto no tiene referencia de la variable utilizada en ese instante. En consecuencia, cualquier modificación en el contenido del parámetro no se verá reflejada en la variable utilizada al momento de la invocación.

Por Referencia: es un parámetro que, en el momento de la invocación del módulo (función/procedimiento), establece una referencia con la variable utilizada en ese instante. En consecuencia, cualquier modificación en el contenido del parámetro se verá reflejada en la variable utilizada al momento de la invocación. Se debe ser cuidadoso al momento de utilizar este tipo de parámetros ya que cualquier modificación en el contenido de éstos afectará al módulo invocante.
Los parámetros por referencia pueden ser a su vez clasificados como parámetros de:
a. Entrada/Salida: Para identificar un parámetro como parámetro de Entrada/Salida, éste deberá estar precedido por la sigla "**ref**" en la declaración e invocación del módulo. Los parámetros tipo arreglo son tratados como parámetros de Entrada/Salida.
b. Salida: Para identificar un parámetro como parámetro de Salida, éste deberá estar precedido por la sigla "**out**" en la declaración e invocación del módulo.

Procedimiento que intercambia el contenido de dos parámetros pasados por referencia.	

Declaración	static void Intercambiar (ref int Param1, ref int Param2)
	{
	int aux;
	aux = Param1;
	Param1 = Param2;
	Param2 = aux;
	}

Utilización	int Valor1, Valor2; Valor1 = 10; Valor2 = 20; Intercambiar(ref Valor1, ref Valor2); Luego de la ejecución de esta última línea, la variable Valor1 contendrá el valor 20, y Valor2 contendrá el valor 10.

Control de Errores

Una excepción es un objeto derivado de System.Exception que se genera cuando en tiempo de ejecución se presenta un error y contiene información detallada del mismo.
El control de errores se realiza a través de la instrucción **try**, la cual permitirá capturar cualquier excepción presentada al intentar ejecutar el bloque de código correspondiente y brindarle un tratamiento adecuado.
Existen una cantidad considerable de clases derivadas de System.Exception que le permitirán al desarrollador tomar medidas específicas de acuerdo al tipo de error detectado, algunas de ellas son:

Excepción	Situación ante la cual se presenta
DivideByZeroException	División por cero
IndexOutOfRangeException	Índice de acceso a elemento de tabla fuera del rango válido (menor que cero o mayor que el tamaño de la tabla)
OverflowException	Desbordamiento
StackOverflowException	Desbordamiento de la pila, generalmente debido a un excesivo número de llamadas recurrentes.

Instrucción Try	

Sintaxis	`try`
	`{`
	`  //Bloque de codigo que se intentara ejecutar`
	`}`
	`catch (Excepcion1)`
	`{`
	`  //Bloque de codigo a ejecutar ante el surgimiento de un error`
	`  //de tipo Excepcion1 en el bloque try`
	`}`
	`catch (Excepcion2)`
	`{`
	`  //Bloque de codigo a ejecutar ante el surgimiento de un error`
	`  //de tipo Excepcion2 en el bloque try`
	`}`
	`.`
	`.`
	`finally`
	`{`
	`  //Opcional:`
	`  //Bloque de codigo a ejecutar luego de los bloques try/catch`
	`}`

Ejemplo	`Try`
	`{`
	`  Console.WriteLine("Resultado es: {0}", n1/n2);`
	`}`
	`catch (DivideByZeroException E)`
	`{`
	`  Console.WriteLine("Error: {0}", E.Message);`
	`}`
	`finally`
	`{`
	`  Console.WriteLine("Operación Finalizada.");`
	`}`

Generación de Excepciones

Existen situaciones en las que es necesario generar una excepción propia del módulo que se esté desarrollando. Para lanzar una excepción se recurre a la utilización de la instrucción ***throw***.
El siguiente ejemplo se presenta solo con fines explicativos, el mismo genera una excepción vinculada con la División por Cero. CSharp cuenta con una excepción para este tipo de situaciones denominada DivideByZeroException.

Ejemplo

```
static void Main(string[] args)
{
   string Cadena;
   float Num1, Num2;
   char Oper;
   float Resultado;

   Console.WriteLine("Ingrese Primer Numero");
   Cadena = Console.ReadLine();
   Num1 = float.Parse(Cadena);

   Console.WriteLine("Ingrese Segundo Numero");
   Cadena = Console.ReadLine();
   Num2 = float.Parse(Cadena);

   Console.WriteLine("Ingrese Operador");
   Cadena = Console.ReadLine();
   Oper = char.Parse(Cadena);

   switch (Oper)
   {
      case '+':
         {
            Resultado = Sumar(Num1,Num2);
            Console.WriteLine("El Resultado es: {0}", Resultado);
            break;
         }
      case '-':
         {
            Resultado = Restar(Num1, Num2);
            Console.WriteLine("El Resultado es: {0}", Resultado);
            break;
         }
      case '*':
         {
            Resultado = Multiplicar(Num1, Num2);
            Console.WriteLine("El Resultado es: {0}", Resultado);
            break;
         }
```

```
      case '/':
        {
          try
          {
            Resultado = Dividir(Num1, Num2);
            Console.WriteLine("El Resultado es: {0}", Resultado);
          }

          catch (Exception e)
          {
            Console.WriteLine("Error: {0}", e.Message);
            Console.WriteLine("Por ayuda recurrir a: {0}", e.HelpLink);
            Console.WriteLine("Software que genero el error: {0}", e.Source);
          }
          break;
        }
    }
    Console.ReadKey();
}
static float Sumar(float N1, float N2)
{
    return N1 + N2;
}
static float Restar(float N1, float N2)
{
    return N1 - N2;
}
static float Multiplicar(float N1, float N2)
{
    return N1 * N2;
}
static float Dividir(float N1, float N2)
{
    if (N2 != 0)
    {
        return N1 / N2;
    }
    else
    {
        Exception Excep = new Exception("Intento de division por cero.");
        Excep.HelpLink = "Consultar Fundamentos Matematicos.";
        Excep.Source = "Programa Calculadora";
        throw Excep;
    }
}
```